



BBDD – Análisis Funcional de Datos

⚙ Estado	Completado
👤 Propietario	🇪🇸 Oscar Cerlanga [Ⓜ] María Paradela [Ⓡ] Raúl Leciñena
☰ Categoría	
☰ Equipo	

1. Requisitos funcionales relacionados con datos

RF1 – Gestión de usuarios

El sistema permitirá registrar, modificar y eliminar usuarios, distinguiendo entre usuario normal y administrador mediante un rol de acceso.

RF2 – Gestión de reservas

El sistema permitirá crear, modificar y cancelar reservas.

RF3 – Gestión de recursos

El sistema permitirá asociar equipos y DJs a una reserva.

RF4 – Generación de presupuestos

El sistema permitirá generar presupuestos asociados a reservas.

RF5 – Generación de facturas

El sistema permitirá generar facturas asociadas a presupuestos aprobados.

RF6 – Registro de pagos

El sistema permitirá registrar uno o varios pagos asociados a una factura.

RF7 – Gestión de contacto

El sistema permitirá recibir y registrar formularios de contacto enviados desde la navegación pública.

RF8 – Datos fiscales del emisor

El sistema almacenará los datos fiscales de la empresa (nombre, CIF, dirección, código postal, localidad, provincia, email, teléfono, IBAN y logotipo) para su inclusión en presupuestos y facturas. El logotipo se gestiona mediante subida a Supabase Storage.

RF9 – Dirección completa del cliente

El sistema recogerá y almacenará la dirección completa del cliente (dirección, código postal, localidad y provincia) tanto en el perfil de usuario como en la reserva, para su inclusión en presupuestos y facturas.

RF10 – Trazabilidad de pagos online

El sistema almacenará la referencia de pago devuelta por la API de pagos (Stripe) para permitir la trazabilidad de cada transacción online, garantizando la idempotencia mediante comprobación previa a la inserción.

RF11 – Gestión de eventos

El sistema permitirá crear, modificar y eliminar eventos públicos (conciertos, actuaciones, etc.) con título, fecha y hora, lugar, artistas, descripción e imagen. Los eventos serán visibles públicamente y gestionables desde el panel de administración.

RF12 – Tokens de acción

El sistema generará tokens de un solo uso para permitir que el cliente acepte o rechace su presupuesto directamente desde el enlace del correo electrónico, sin necesidad de iniciar sesión. Cada token referencia un presupuesto concreto, almacena el tipo de acción, registra si ya ha sido usado y caduca a las 48 horas.

RF13 – Respuesta a contactos

El sistema permitirá al administrador responder a los formularios de contacto recibidos directamente desde el panel de administración. La respuesta se enviará automáticamente al usuario por correo electrónico mediante N8N y quedará registrada en el sistema junto con el estado de respondido.

RF14 – Automatizaciones N8N

El sistema integrará automatizaciones mediante N8N para el envío de correos electrónicos en los eventos clave del ciclo de vida de una reserva: creación de presupuesto, aceptación y rechazo por token, confirmación del administrador, registro de pagos parciales, factura pagada, recepción de contacto y respuesta al contacto.

RF15 – Integración de pagos online (Stripe)

El sistema integrará la pasarela de pagos Stripe para permitir el pago online de facturas. El cliente podrá realizar pagos totales o parciales mediante tarjeta. El backend validará el importe, creará un PaymentIntent y procesará el resultado mediante webhook firmado.

RF16 – Modo de mantenimiento

El sistema dispondrá de un modo de mantenimiento activable mediante variable de entorno, que devuelve un error 503 en todas las rutas de la API y redirige al usuario a una página informativa desde el frontend.

2. Reglas de negocio aplicadas a los datos

RN1. Una reserva puede componerse de cero, uno o varios equipos.

RN2. Una reserva puede componerse de cero, uno o varios DJs.

RN3. Una reserva deberá incluir al menos un equipo o un DJ.

RN4. Una reserva puede generar como máximo un presupuesto.

RN5. Un presupuesto puede generar como máximo una factura.

RN6. La base imponible del presupuesto se calculará como la suma de los subtotales de sus líneas de detalle.

RN7. Toda factura debe estar asociada a un presupuesto.

RN8. Una factura puede tener uno o varios pagos.

RN9. El estado de pago de una factura se determina por la suma de los importes de los pagos asociados.

RN10. Las contraseñas se almacenarán mediante hash criptográfico seguro. La gestión de la autenticación la realiza Supabase Auth; no se almacena ningún hash en la tabla usuario.

- RN11.** La disponibilidad de equipos y DJs se determinará en función de las reservas existentes y sus fechas.
- RN12.** No podrán existir reservas solapadas en el tiempo para un mismo equipo o DJ.
- RN13.** El total del presupuesto y de la factura se calculará como $\text{base_imponible} + \text{IVA del 21\%}$.
- RN14.** La entidad Ubicación podrá evolucionar a una entidad independiente si el sistema requiere mayor normalización o reutilización de direcciones.
- RN15.** Un contacto deberá almacenar nombre y apellidos, email, título del problema, tipo de consulta, descripción y fecha.
- RN16.** Un contacto podrá registrarse sin necesidad de que exista una sesión iniciada en el sistema.
- RN17.** La entidad Contacto no formará parte del flujo de reserva ni de la estructura económica del sistema.
- RN18.** El tipo de contacto quedará restringido a un conjunto cerrado de valores: duda, incidencia, opinión u otro.
- RN19.** El método de pago se almacena como texto libre (VARCHAR) para admitir los valores devueltos por la API de pagos externa (Stripe) así como los registrados manualmente por el administrador. Los valores posibles en el sistema actual son: `stripe`, `efectivo` y `transferencia`.
- RN20.** La fecha de inicio de una reserva deberá ser anterior a su fecha de fin.
- RN21.** Las cantidades de equipos asociados a una reserva y en los detalles de presupuesto deberán ser estrictamente positivas.
- RN22.** Los importes económicos (precios, subtotales, base imponible, total e importe de pago) no podrán ser negativos a menos que se apliquen devoluciones.
- RN23.** Toda reserva deberá contener los datos completos del cliente (DNI/NIE/CIF, nombre, email, teléfono, dirección, código postal, localidad y provincia), ya sea copiados automáticamente desde el usuario registrado o introducidos manualmente por el administrador.
- RN24.** Los datos del cliente almacenados en la reserva servirán como origen para la generación de presupuestos y facturas, garantizando su disponibilidad incluso si el usuario elimina su cuenta o si simplemente no la tiene creada (reserva creada por el administrador).

RN25. Los datos fiscales de la empresa serán únicos en el sistema y se utilizarán como datos del emisor en presupuestos y facturas.

RN26. La entidad Datos_Empresa almacenará el IBAN de la empresa para su inclusión en presupuestos y facturas cuando el método de pago sea transferencia bancaria.

RN27. Los pagos realizados a través de la plataforma online generarán una referencia de pago devuelta por la API que deberá almacenarse para permitir la trazabilidad de la transacción. Los pagos en efectivo no generarán referencia de pago (valor nulo).

RN28. Un token de acción solo puede usarse una vez. Una vez marcado como usado, cualquier intento posterior de utilizarlo será rechazado por el sistema con una respuesta informativa al usuario.

RN29. Un token de acción caduca a las 48 horas desde su creación. Los tokens caducados no podrán ejecutar ninguna acción.

RN30. El estado `aceptado_cliente` en un presupuesto indica que el cliente ha aceptado pero el administrador aún no ha confirmado. El estado final `aceptado` solo lo puede establecer el administrador desde el panel.

RN31. Un evento puede existir sin relación con ninguna reserva ni usuario del sistema. Es una entidad de información pública independiente del flujo de reservas.

RN32. Un contacto puede ser respondido por el administrador exactamente una vez. Una vez respondido, el campo `respondido` queda a true y no puede revertirse desde la interfaz.

RN33. La comprobación de idempotencia en pagos online se realiza verificando si ya existe un registro con el mismo valor de `referencia_pago` antes de insertar, evitando duplicados por reenvíos del webhook de Stripe.